Scholaris - Frontend to Backend Handoff Report

Project: Scholaris (SDG 4 - Quality Education) **Course:** BIT1123/BISE2093/DIT1113 Object Oriented Programming - Final Project **Frontend by:** [Your name] **This document is for:** the teammate implementing the backend / matching logic

---

1. What this app does

Scholaris is a desktop app (JavaFX) that takes a student's profile (age, GPA, nationality, field of study) and shows them a list of scholarships they're likely to qualify for, with a detail page per scholarship.

The **frontend is fully built and running**. It currently displays 3 hardcoded scholarships so the UI could be built and demoed without waiting on the backend. Your job is to replace that hardcoded data with real logic.

2. How to run the project

```
mvn clean javafx:run
```

Requires JDK 17+ and Maven. No manual JavaFX SDK download needed - Maven pulls it automatically via the `javafx-maven-plugin` in `pom.xml`.

3. Screen flow

```
LandingScreen  ->  ProfileScreen  ->  MatchesScreen  ->
ScholarshipDetailScreen
   (hero/CTA)     (profile form)    (list of cards)      (one
scholarship,

Prev/Next in header)
```

All screens are plain Java classes extending `VBox` (no FXML). `Main.java` holds a single `StackPane` and swaps screens in and out of it - `Main.switchScreen(new SomeScreen())`.

4. File structure

```
src/main/java/com/scholaris/
  Main.java                        - app entry point, screen
switching
  NavBar.java                      - shared floating header
  LandingScreen.java               - hero screen
  ProfileScreen.java               - profile input form  <-- you
need this data
  MatchesScreen.java               - shows matched scholarships
  ScholarshipDetailScreen.java     - one scholarship's full detail
  Scholarship.java                 - data model            <-- see
below
  ScholarshipRepository.java       - interface              <-- YOU
implement this
  StaticScholarshipRepository.java - TEMP hardcoded data     <-- you
```

```
will delete/replace this
src/main/resources/app.css         - all styling (don't need to
touch this)
```

5. The ONE integration point you need: `ScholarshipRepository`

The UI never touches hardcoded data directly. Every screen that needs scholarship data goes through this interface:

```java
public interface ScholarshipRepository {
    List<Scholarship> getAllScholarships();
}
```

Right now `MatchesScreen.java` does this:

```java
ScholarshipRepository repo = new StaticScholarshipRepository();
List<Scholarship> scholarships = repo.getAllScholarships();
```

What you need to do:

1. Create a new class, e.g. `MatchingScholarshipRepository implements ScholarshipRepository`.
2. Inside `getAllScholarships()`, put your real logic: read scholarships from a file (satisfies the "File Handling" rubric item), filter/rank them against the student's profile (GPA, nationality, field of study, age), and return the matched list as `Scholarship` objects.
3. In `MatchesScreen.java`, swap the one line:

   ```java
   ScholarshipRepository repo = new StaticScholarshipRepository();
   ```

   to:

   ```java
   ScholarshipRepository repo = new
MatchingScholarshipRepository(studentProfile);
   ```

4. Delete `StaticScholarshipRepository.java` once yours is working.

Nothing else in the UI needs to change - `MatchesScreen` and `ScholarshipDetailScreen` only ever call `getAllScholarships()` and read the `Scholarship` objects returned.

6. The data model: `Scholarship`

```java
public class Scholarship {
    private final String title;
    private final String description;
    private final List<String> tags;
    private final String overview;
    private final List<String> eligibility;
    private final String websiteUrl;
    // constructor + getters only, no setters (encapsulation)
}
```

If you need more fields for your matching logic (e.g. `minGpa`, `eligibleNationalities`, `eligibleFields`), add them here as private fields with getters. The UI will just ignore fields it doesn't display.

7. Getting the student's profile data to you

`ProfileScreen.java` currently has 4 `TextFields` (age, GPA, nationality, field of study) but the "Match Me" button just navigates to `MatchesScreen` without passing the values anywhere - marked with a `// TODO(backend)` comment in the code.

Suggested fix (I can do this once you tell me your class name/shape):

1. You create a simple `StudentProfile` class:

```
public class StudentProfile {
    private final int age;
    private final double gpa;
    private final String nationality;
    private final String fieldOfStudy;
    // constructor + getters
}
```

2. In `ProfileScreen`, the button handler builds one of these from the text fields and passes it into `MatchesScreen`'s constructor, which passes it into your `MatchingScholarshipRepository`.

Let me know once your class exists and I'll wire the button up.

8. Rubric requirements this project still needs (mostly on your side)

Requirement	Status	Suggested owner
Classes and Objects (6-8 min)	Partial - frontend has 9	Combined total should easily clear 6-8
Encapsulation	Done in `Scholarship` (private fields, getters only)	-
Inheritance (at least 1 hierarchy)	**Not yet done**	Backend - see below
Polymorphism (overriding, runtime)	**Not yet done**	Backend - see below
Abstraction	`ScholarshipRepository` interface	Done, both sides use it
Collections (ArrayList/HashMap)	`List` used in UI already	Backend should use these for matching/filtering
File I/O	**Not yet done**	Backend - read scholarships from a file

Suggested inheritance/polymorphism design (for the rubric, your call)

Turn `Scholarship` into an abstract class with subclasses, each overriding an eligibility check - this is the natural place for those OOP marks:

```
public abstract class Scholarship {
    // shared fields: title, description, tags, overview,
eligibility, url
    public abstract boolean isEligible(StudentProfile profile);
}

public class MeritScholarship extends Scholarship {
```

```
        private double minGpa;
        @Override
        public boolean isEligible(StudentProfile profile) {
            return profile.getGpa() >= minGpa;
        }
    }

    public class NeedBasedScholarship extends Scholarship {
        // different eligibility rule
        @Override
        public boolean isEligible(StudentProfile profile) { ... }
    }
```

Your `MatchingScholarshipRepository` would then call `scholarship.isEligible(profile)` polymorphically while filtering the file's data - this satisfies inheritance, polymorphism, and abstraction in one clean piece of your code, and the frontend won't need any changes since it only ever reads the getters that already exist.

9. Suggested file format for scholarship data

A simple text or CSV file works fine for the File Handling requirement, e.g. `scholarships.txt`:

```
title=The Pioneer STEM Tech Fellowship
tags=STEM,Technology,Undergrad
minGpa=3.5
fields=Computer Science,Engineering
description=Designed for undergraduate computer science majors...
overview=The Pioneer STEM Tech Fellowship exists to...
eligibility=Currently enrolled full-time...|Minimum GPA of 3.5...
url=https://example.com/pioneer-stem-fellowship
---
title=Global Leaders Academic Foundation
...
```

Or JSON/CSV if you're more comfortable with those - doesn't matter to the UI as long as your repository class parses it into `Scholarship` objects.

10. Contact points if something breaks

- If `mvn clean javafx:run` fails after you add your class, check the error for a missing import or a method signature mismatch against `ScholarshipRepository`.
- If the app runs but shows no scholarships, your `getAllScholarships()` is probably returning an empty list - add a `System.out.println` to debug.
- Don't touch `app.css`, `NavBar.java`, or any `*Screen.java` files unless you're changing the profile-data wiring described in Section 7 - the visuals are already matched to the Figma design and any CSS class name changes will break the styling.